

## ARCHITECTURE FONDATRICE

# Gestion des Utilisateurs v2

Le socle qui supporte toute la vision Leopardo RH

**Existant modernisé**

user\_lookups · RBAC · plans

**Nouveaux besoins**

Invitations · Devices · 2FA

**Progressive**

Stades · Leo · Onboarding

**Évolutif**

API publique · Marketplace

**Ce qui existe :** RBAC 7 rôles · user\_lookups · TenantMiddleware · Multitenancy dual · Audit logs

**Ce qui manque :** Invitations email · Sessions devices · 2FA · Stades apprentissage · Délégation

**Ce qui évolue :** employees.metadata JSONB · plans.features extensibles · Tokens révocables par device

**Horizon Phase 2 :** Permissions granulaires · API publique · Rôles custom · Multi-sociétés

**Chapitres :** 10 · Modèle de données · RBAC · Auth · Sessions · Invitations · API · Tâches

**7**

Rôles RBAC

**2**

Modes tenancy

**48**

Tâches v2

**∞**

Évolutif

# TABLE DES MATIÈRES

|           |                                                       |                                                        |
|-----------|-------------------------------------------------------|--------------------------------------------------------|
| <b>01</b> | <b>Analyse honnête — état actuel vs vision</b>        | Ce qui est solide · Ce qui manque · Les 5 décisions    |
| <b>02</b> | <b>Modèle de données v2 — évolution complète</b>      | Nouvelles tables · Colonnes ajoutées · Index           |
| <b>03</b> | <b>Les 7 rôles — RBAC modernisé</b>                   | Rôles enrichis · Matrice complète · Délégation         |
| <b>04</b> | <b>Cycle de vie d'un utilisateur</b>                  | De l'invitation à l'archivage · Tous les états         |
| <b>05</b> | <b>Authentification rock-solid</b>                    | Login · 2FA · Sessions · Devices · Révocation          |
| <b>06</b> | <b>Système d'invitations</b>                          | Inviter un employé par email · Flow complet · Sécurité |
| <b>07</b> | <b>Progressive Disclosure &amp; Stades</b>            | Leo intégré · Apprentissage · Adaptation par stade     |
| <b>08</b> | <b>Plans &amp; Features — architecture extensible</b> | Plan JSONB · Feature flags · Upgrade/downgrade         |
| <b>09</b> | <b>Audit trail &amp; Sécurité</b>                     | Qui a fait quoi · Quand · Depuis où · Alertes          |
| <b>10</b> | <b>Toutes les tâches — USR-01 à USR-48</b>            | 4 sprints · DB · Laravel · Flutter · Priorités         |
| <b>11</b> | <b>Horizon Phase 2 — Permissions granulaires</b>      | Rôles custom · Délégation fine · API publique          |

## CH.01

## ANALYSE HONNÊTE — ÉTAT ACTUEL VS VISION

Ce qui est solide · Ce qui manque · Les 5 décisions

01

Ce document ne repart pas de zéro. La base actuelle est solide : `user_lookups`, `TenantMiddleware dual-mode`, `RBAC 7 rôles`, `Global Scope`, `audit_logs`. Ce qui manque ce sont les couches qui permettent à la plateforme de vivre, grandir et s'adapter — `invitations`, `sessions`, `2FA`, `stades d'apprentissage`, `délégation`.

## Inventaire honnête — ce qui existe et fonctionne bien

| Composant                                               | État       | Qualité                                                                             | Action v2                                                                                                         |
|---------------------------------------------------------|------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>user_lookups</code> (schéma public)               | Implémenté | Excellente — 1 requête pour trouver n'importe quel email                            | Ajouter <code>device_type</code> dans la clé composite pour les sessions                                          |
| <code>TenantMiddleware</code> (dual-mode shared/schema) | Implémenté | Excellente — <code>search_path</code> dynamique                                     | Ajouter le stade <code>learning_stage</code> dans l'injection de contexte                                         |
| <code>Global Scope company_id</code> (mode shared)      | Implémenté | Solide — automatique sur chaque query                                               | Ne pas toucher — c'est le bon pattern                                                                             |
| <code>RBAC 7 rôles</code> (role + manager_role)         | Documenté  | Bonne — matrice complète dans le doc                                                | Enrichir <code>employees.metadata</code> pour les stades et préférences                                           |
| <code>audit_logs</code> (actor_type polymorphique)      | Implémenté | Bonne — gère <code>super_admin</code> + <code>employee</code> + <code>system</code> | Ajouter <code>ip_address</code> et <code>user_agent</code> pour la sécurité                                       |
| <code>employee_devices</code> (FCM tokens)              | Implémenté | Bonne pour les notifs push                                                          | Étendre pour les sessions multi-device et la révocation                                                           |
| Sanctum tokens                                          | Implémenté | Standard Laravel                                                                    | Ajouter <code>last_used_at</code> , <code>device_name</code> , <code>platform</code> pour la gestion des sessions |
| <code>plans.features JSONB</code>                       | Implémenté | Bonne base                                                                          | Standardiser les clés et documenter tous les feature flags                                                        |

## Ce qui manque — les lacunes bloquantes pour la vision

| Lacune                            | Impact sur la vision                                                                                                                                       | Priorité v2 |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Invitations par email             | Un manager ne peut pas inviter un employé en lui envoyant un lien sécurisé. L'admin doit créer les comptes manuellement. Bloque l'onboarding self-service. | Critique    |
| Gestion des sessions multi-device | Un token Sanctum créé sur iPhone est valide sur tous les devices. Impossible de révoquer une session spécifique (ex: téléphone perdu). Risque sécurité.    | Critique    |

| Lacune                                 | Impact sur la vision                                                                                                                                                   | Priorité v2 |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| 2FA (Two-Factor Authentication)        | Aucune vérification en deux étapes. Compte compromis = accès total aux données financières et RH.                                                                      | Haute       |
| Stades d'apprentissage (LearningStage) | L'APV v2 et le Design System v3 prévoient 5 stades (Découverte → Ambassadeur). La colonne <code>employees.metadata.learning_stage</code> n'est pas encore implémentée. | Haute       |
| Système de délégation temporaire       | Un manager en vacances ne peut pas déléguer ses approbations à un collègue. Bloque l'usage en entreprise réelle.                                                       | Haute       |
| Révocation de tokens par device        | Impossible de déconnecter un device spécifique. Sanctum révoque tous les tokens ou aucun.                                                                              | Haute       |
| Politique de mot de passe              | Aucune règle de complexité, aucune expiration, aucun historique. Non-conforme aux exigences de sécurité entreprise.                                                    | Haute       |
| Rate limiting login ciblé              | Le rate limiting existe mais n'est pas différencié par email — un attaquant peut tenter 60 mots de passe différents sur le même compte avant d'être bloqué.            | Haute       |
| Rôles custom (Phase 2)                 | Les 7 rôles sont fixés dans le code. Un client Enterprise ne peut pas créer son propre rôle 'Superviseur Sécurité'. Bloque la customisation entreprise.                | Phase 2     |

## Les 5 décisions architecturales — à prendre maintenant

- 1. employees.metadata JSONB est le registre vivant de l'utilisateur.** Toutes les données qui évoluent fréquemment (`learning_stage`, `app_actions_count`, `preferred_chips_order`, `first_seen_screens`, `preferred_language`) vont dans ce JSONB. Pas de nouvelle colonne pour chaque feature — sauf quand la donnée est critique et indexable.
- 2. Les sessions Sanctum sont augmentées, pas remplacées.** On n'abandonne pas Sanctum. On ajoute `device_name`, `platform`, `last_used_at`, `last_ip_address` aux `personal_access_tokens` (via la table `token_metadata`) pour permettre la gestion multi-device.
- 3. Les invitations sont des tokens éphémères, pas des comptes provisoires.** Un token d'invitation existe dans une table `invitations` séparée. Il est utilisé une seule fois pour créer le compte. Il expire après 7 jours. Jamais de compte utilisateur 'en attente'.
- 4. La délégation est temporelle et scoped.** Le Manager A peut déléguer au Manager B son droit d'approbation des congés du 1er au 15 août. La délégation porte sur une permission précise (pas le rôle entier) et une période définie. Table `delegations`.
- 5. Le stade d'apprentissage pilote l'interface.** `LearningStageService` calcule le stade depuis `metadata.app_actions_count`. L'API retourne le stade dans `/api/v1/auth/me`. Flutter et Blade l'utilisent pour le progressive disclosure. Un seul service, une seule source de vérité.

## CH.02

## MODÈLE DE DONNÉES V2 — ÉVOLUTION COMPLÈTE

Nouvelles tables · Colonnes ajoutées · Index critiques

02

## Évolution de la table employees — colonnes ajoutées

La table employees est dans le schéma tenant (shared ou schema selon le plan). Les colonnes suivantes sont ajoutées en migration. Toutes sont backward-compatible.

Migration : colonnes v2 sur employees

-- Migration : add\_v2\_columns\_to\_employees

```
ALTER TABLE employees ADD COLUMN IF NOT EXISTS
  preferred_language CHAR(2) NULL REFERENCES languages(code),
  -- Langue préférée (NULL = utilise company.language)
```

```
ALTER TABLE employees ADD COLUMN IF NOT EXISTS
```

```
  metadata JSONB NOT NULL DEFAULT '{}',
  -- Clés standards du JSONB :
  -- app_actions_count INT      (compteur pour LearningStage)
  -- learning_stage VARCHAR    'discovery'|'routine'|'mastery'|'expert'|'ambassador'
  -- first_seen_screens []      (liste des écrans vus pour la première fois)
  -- preferred_chips_order []   (ordre des Quick Actions chips – Phase 2)
  -- last_seen_version VARCHAR (pour le changelog in-app)
  -- onboarding_completed BOOL (checklist initiale terminée)
```

```
ALTER TABLE employees ADD COLUMN IF NOT EXISTS
```

```
  two_fa_enabled BOOL NOT NULL DEFAULT false,
  two_fa_secret VARCHAR(64) NULL, -- Chiffré AES-256
  two_fa_backup_codes TEXT NULL,  -- Array JSON de codes de secours chiffrés
```

```
ALTER TABLE employees ADD COLUMN IF NOT EXISTS
```

```
  password_changed_at TIMESTAMPTZ NULL,
  failed_login_count SMALLINT NOT NULL DEFAULT 0,
  locked_until TIMESTAMPTZ NULL;
```

-- Index sur metadata pour les queries fréquentes

```
CREATE INDEX idx_employees_learning_stage ON employees
  USING gin ((metadata->>'learning_stage'));
CREATE INDEX idx_employees_actions_count ON employees
  ((metadata->>'app_actions_count')::int);
```

## Nouvelle table : invitations

```

invitations – nouvelle table schéma public

-- Table invitations (schéma PUBLIC – cross-tenant)
-- Pourquoi public : le token doit être accessible avant que le tenant soit connu
CREATE TABLE invitations (
    id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_id        UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
    email              VARCHAR(150) NOT NULL,
    role               VARCHAR(20)  NOT NULL, -- 'employee' | 'manager'
    manager_role       VARCHAR(30)  NULL,     -- 'rh' | 'comptable' | 'superviseur' | ...
    token              VARCHAR(64)  NOT NULL UNIQUE, -- Token aléatoire sécurisé (Str::random(64))
    token_expires_at   TIMESTAMPTZ NOT NULL, -- NOW() + 7 jours
    invited_by         INT           NOT NULL, -- employees.id du manager qui invite
    invited_by_name     VARCHAR(100) NOT NULL, -- Dénormalisé pour affichage sans JOIN
    accepted_at        TIMESTAMPTZ  NULL,     -- NULL = pas encore accepté
    revoked_at         TIMESTAMPTZ  NULL,     -- NULL = actif
    created_at         TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- Contrainte : un email ne peut avoir qu'une invitation active par company
CREATE UNIQUE INDEX idx_invitations_active ON invitations (company_id, email)
    WHERE accepted_at IS NULL AND revoked_at IS NULL AND token_expires_at > NOW();

```

### Nouvelle table : token\_metadata (sessions enrichies)

```

token_metadata – sessions enrichies

-- Extension de personal_access_tokens de Sanctum
-- Sanctum ne permet pas d'ajouter des colonnes à sa table – on crée une table liée
CREATE TABLE token_metadata (
    token_id          BIGINT PRIMARY KEY REFERENCES personal_access_tokens(id) ON DELETE CASCADE,
    device_name        VARCHAR(150) NOT NULL, -- 'iPhone 15 Pro de Ahmed'
    platform           VARCHAR(20)  NOT NULL, -- 'ios' | 'android' | 'web'
    app_version        VARCHAR(20)  NULL,     -- '1.2.3'
    os_version         VARCHAR(30)  NULL,     -- 'iOS 17.4.1'
    last_ip            INET         NULL,     -- Dernière IP connue
    last_country       CHAR(2)      NULL,     -- GeoIP2 – pays de connexion
    last_city          VARCHAR(100) NULL,     -- Ville de connexion
    last_used_at       TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    created_at         TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- Index pour la vue 'Mes sessions actives'
CREATE INDEX idx_token_metadata_last_used ON token_metadata (last_used_at DESC);

-- Vue utilitaire : sessions actives d'un employé
CREATE VIEW active_sessions AS
SELECT pat.id, pat.tokenable_id as employee_id,
    tm.device_name, tm.platform, tm.last_ip,
    tm.last_used_at, tm.last_country, pat.created_at
FROM personal_access_tokens pat
JOIN token_metadata tm ON tm.token_id = pat.id
WHERE pat.expires_at IS NULL OR pat.expires_at > NOW();

```

### Nouvelle table : delegations (délégation temporaire)

```
delegations – délégation temporaire de permissions
-- Table delegations (schéma TENANT – par company)
CREATE TABLE delegations (
  id                INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  delegator_id      INT NOT NULL REFERENCES employees(id), -- Qui délègue
  delegate_id       INT NOT NULL REFERENCES employees(id), -- À qui
  permission_scope  VARCHAR(50) NOT NULL, -- 'leave_approvals' | 'expense_approvals' | 'all'
  starts_at         DATE NOT NULL,        -- Début de la délégation
  ends_at           DATE NOT NULL,        -- Fin de la délégation (inclusif)
  reason            VARCHAR(255) NULL,     -- 'Congé annuel du manager'
  is_active         BOOL NOT NULL DEFAULT true,
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT chk_dates CHECK (ends_at >= starts_at),
  CONSTRAINT chk_not_self CHECK (delegator_id != delegate_id)
);

-- Index pour résoudre rapidement les délégations actives
CREATE INDEX idx_delegations_active ON delegations (delegate_id, is_active)
  WHERE is_active = true AND ends_at >= CURRENT_DATE;
```

### Nouvelle table : password\_history

```
password_history – historique mots de passe
-- Table password_history (schéma TENANT)
-- Empêcher la réutilisation des 5 derniers mots de passe
CREATE TABLE password_history (
  id                INT PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
  employee_id       INT NOT NULL REFERENCES employees(id) ON DELETE CASCADE,
  password_hash     VARCHAR(255) NOT NULL, -- Hash bcrypt du mot de passe
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Garder seulement les 5 derniers par employé (trigger ou service)
CREATE INDEX idx_pwd_history_employee ON password_history (employee_id, created_at DESC);
```

## CH.03

## LES 7 RÔLES — RBAC MODERNISÉ

Rôles enrichis · Matrice complète · Délégation · Héritage

03

Le RBAC existant est bon. On ne le casse pas. On l'enrichit avec deux nouvelles dimensions : (1) les stades d'apprentissage qui modulent ce qui est visible selon l'expérience de l'utilisateur, et (2) la délégation temporaire qui permet à un rôle de prêter une permission précise à un autre rôle.

## Les 7 rôles — cartes détaillées

## SUPER ADMIN

Éditeur SaaS — hors tenant

## ✓ PEUT

- Accès total à toutes les entreprises
- Créer/suspendre/réactiver des companies
- Gérer les plans et les features
- Voir les métriques SaaS globales
- Accéder en lecture aux schémas tenant

## ✗ NE PEUT PAS

- Pointer une présence
- Accéder aux données RH courantes
- Approuver des congés
- Créer des factures clients

## MANAGER PRINCIPAL

Scope : entreprise entière

## ✓ PEUT

- Voir et modifier tous les employés
- Toutes les approbations (congés, dépenses, avances)
- Configurer les modules et les settings
- Inviter de nouveaux utilisateurs
- Corriger les pointages
- Voir les données financières

## ✗ NE PEUT PAS

- Pointer sa propre présence (il est manager)
- Accéder à une autre entreprise

## MANAGER RH

Scope : tous les employés, 0 finance

## ✓ PEUT

- Gérer les fiches employés (créer, modifier, archiver)
- Gérer les congés et absences pour tous
- Voir les pointages de tous
- Corriger les pointages
- Générer les rapports RH

## ✗ NE PEUT PAS

- Voir les salaires et les IBAN (sauf les siens)
- Accéder aux modules Finance et Caméras
- Approuver les dépenses

## MANAGER COMPTABLE

Scope : finance + paie, 0 RH opérationnel

## ✓ PEUT

- Voir noms, matricules, salaires, IBAN de tous
- Accès complet au module Finance
- Générer les fiches de paie
- Valider les virements
- Créer des bons de commande fournisseurs

## ✗ NE PEUT PAS

- Voir les données personnelles (adresse, téléphone)
- Approuver des congés
- Corriger des pointages
- Gérer les fiches employés



**MANAGER DÉPARTEMENT**

Scope : son département uniquement

**✓ PEUT**

- Voir les employés de son département
- Approuver congés et absences de son équipe
- Corriger les pointages de son équipe
- Créer des tâches pour son équipe
- Voir le planning de son département

**✗ NE PEUT PAS**

- Voir les employés d'autres départements
- Accéder aux modules Finance ou Caméras
- Modifier les paramètres de l'entreprise

**SUPERVISEUR**

Scope : son équipe assignée uniquement

**✓ PEUT**

- Voir les employés de son équipe
- Approuver congés de son équipe
- Voir les pointages de son équipe
- Valider les tâches de son équipe

**✗ NE PEUT PAS**

- Corriger des pointages
- Créer des employés
- Voir des données financières
- Voir d'autres équipes

**EMPLOYÉ**

Scope : ses propres données uniquement

**✓ PEUT**

- Pointer son arrivée et son départ
- Voir son propre historique de pointage
- Soumettre des demandes de congé
- Voir ses fiches de paie
- Modifier son profil (photo, téléphone, MDP)

**✗ NE PEUT PAS**

- Voir les données d'autres employés
- Accéder à aucun module de gestion
- Corriger son propre pointage

**La délégation — comment ça fonctionne en code**

DelegationService.php — résolution des permissions déléguées

```
// app/Services/DelegationService.php
class DelegationService {
    // Résoudre qui peut approuver quoi aujourd'hui
    public function canApprove(Employee $actor, string $permission): bool {
        // 1. Le rôle direct de l'acteur
        if ($this->roleAllows($actor, $permission)) return true;
        // 2. Une délégation active en sa faveur
        return Delegation::query()
            ->where('delegate_id', $actor->id)
            ->where('is_active', true)
            ->where('starts_at', '<=', today())
            ->where('ends_at', '>=', today())
            ->where(fn($q) =>
                $q->where('permission_scope', $permission)
                ->orWhere('permission_scope', 'all'))
            ->exists();
    }
    // Dans les Gates / Policies :
    // Gate::define('approve-leave', fn(Employee $e) =>
    //     app(DelegationService::class)->canApprove($e, 'leave_approvals')
    // );
}
```



CH.04

CYCLE DE VIE D'UN UTILISATEUR

De l'invitation à l'archivage · Tous les états · Toutes les transitions

04

Les 6 états d'un utilisateur



Chaque état a des permissions précises et des transitions autorisées.

| État             | Qui peut y placer                      | Actions autorisées                         | Données préservées ?  | user_lookups                     |
|------------------|----------------------------------------|--------------------------------------------|-----------------------|----------------------------------|
| INVITÉ (pending) | Manager Principal / RH                 | Accepter l'invitation (action de l'invité) | —                     | Non — créé lors de l'acceptation |
| ACTIF            | Système (après acceptation invitation) | Toutes les actions de son rôle             | Oui                   | Oui                              |
| INACTIF          | Calculé automatiquement (cron)         | Login toujours possible — accès complet    | Oui                   | Oui                              |
| SUSPENDU         | Manager Principal uniquement           | Login impossible — 403 sur tout            | Oui — gel des données | Oui — email réservé              |
| ARCHIVÉ          | Manager Principal uniquement           | Aucune — compte mort                       | Oui — RGPD compliant  | Oui — email réservé indéfiniment |
| RÉACTIVÉ         | Manager Principal uniquement           | Identiques à ACTIF                         | Déjà préservées       | Déjà présent                     |

Le flow d'invitation — détail technique complet

## Flow invitation complet — 3 étapes

## # ÉTAPE 1 — Manager crée l'invitation

POST /api/v1/employees/invite

Body: { email, role, manager\_role, department\_id (optionnel) }

- Vérifier : email pas déjà dans user\_lookups
- Vérifier : pas d'invitation active pour cet email dans cette company
- Créer invitation { token=Str::random(64), expires\_at=now()+7days }
- Envoyer email : 'Vous avez été invité par [nom manager] à rejoindre [company]'
- Lien : leopardo-rh.com/invitation?token=XXXX

## # ÉTAPE 2 — L'invité clique sur le lien

GET /invitation?token=XXXX (landing page publique)

- Vérifier token valide ET non expiré ET non révoqué
- Afficher : 'Bienvenue chez [company]. Créez votre mot de passe.'
- Pré-remplir l'email (non modifiable)

## # ÉTAPE 3 — L'invité crée son compte

POST /api/v1/invitations/{token}/accept

Body: { name, password, phone (optionnel) }

- DB::transaction() {
  1. SET search\_path TO {company.schema\_name}
  2. INSERT INTO employees { name, email, role, manager\_role, password\_hash }
  3. INSERT INTO user\_lookups { email, company\_id, employee\_id, role }
  4. UPDATE invitations SET accepted\_at = NOW() WHERE token = ?
  5. Créer le token Sanctum + token\_metadata { device\_name, platform }
  6. Incrémenter company.employee\_count (si colonne existe)}
- Retourner { token, employee, company, plan }
- Flutter : stocker token, initialiser Leo avec stade 'discovery'

CH.05

## AUTHENTIFICATION ROCK-SOLID

Login · 2FA · Sessions · Devices · Révocation

05

*La sécurité de l'authentification est non-négociable. Un accès non autorisé aux données d'une PME (salaires, IBAN, présences) peut avoir des conséquences légales pour Leopardo RH. Chaque décision de sécurité ici est intentionnelle.*

### Flow de login enrichi v2

```
AuthController@login — flow complet v2
// app/Http/Controllers/Api/V1/AuthController.php — login()
public function login(LoginRequest $request): JsonResponse {
    $email    = $request->email;
    $password = $request->password;

    // 1. Rate limiting ciblé par email (pas juste par IP)
    $key = 'login:' . Str::lower($email);
    if (RateLimiter::tooManyAttempts($key, 5)) {
        return response()->json([
            'error_code' => 'TOO_MANY_ATTEMPTS',
            'retry_after' => RateLimiter::availableIn($key),
        ], 429);
    }

    // 2. Lookup rapide (1 requête schéma public)
    $lookup = UserLookup::find($email);
    if (!$lookup) { RateLimiter::hit($key); return $this->invalidCredentials(); }

    // 3. Charger l'employé dans le bon schéma tenant
    $company = Company::find($lookup->company_id);
    DB::statement('SET search_path TO ' . $company->schema_name . ', public');
    $employee = Employee::find($lookup->employee_id);

    // 4. Vérifier le compte n'est pas suspendu/archivé
    if ($employee->status === 'suspended') return $this->accountSuspended();
    if ($employee->status === 'archived')  return $this->accountArchived();

    // 5. Vérifier le compte n'est pas verrouillé (brute-force)
    if ($employee->locked_until && $employee->locked_until > now()) {
        return response()->json(['error_code' => 'ACCOUNT_LOCKED'], 403);
    }

    // 6. Vérifier le mot de passe
    if (!Hash::check($password, $employee->password_hash)) {
        RateLimiter::hit($key);
        $employee->increment('failed_login_count');
        if ($employee->failed_login_count >= 10) {
            $employee->update(['locked_until' => now()->addHours(2)]);
        }
        return $this->invalidCredentials();
    }

    // 7. 2FA si activé
    if ($employee->two_fa_enabled) {
        // Retourner un pre-auth token (pas encore un vrai token)
        $preToken = cache()->put('2fa:' . $employee->id, Str::uuid(), 300);
        return response()->json(['requires_2fa' => true, 'pre_token' => $preToken]);
    }

    // 8. Créer le token Sanctum + token_metadata
```

## Gestion des sessions — liste et révocation par device

Sessions – liste et révocation ciblée

```
// GET /api/v1/auth/sessions – Liste les sessions actives de l'utilisateur
public function sessions(Request $request): JsonResponse {
    $sessions = $request->user()->tokens()
        ->join('token_metadata', 'personal_access_tokens.id', '=', 'token_metadata.token_id')
        ->select('personal_access_tokens.id', 'token_metadata.*')
        ->orderBy('token_metadata.last_used_at', 'desc')
        ->get();

    return response()->json($sessions->map(fn($s) => [
        'id' => $s->id,
        'device_name' => $s->device_name,
        'platform' => $s->platform,
        'last_used_at' => $s->last_used_at,
        'last_ip' => $s->last_ip,
        'last_country' => $s->last_country,
        'is_current' => $s->id === $request->user()->currentAccessToken()->id,
    ]));
}

// DELETE /api/v1/auth/sessions/{token_id} – Révoquer une session spécifique
public function revokeSession(Request $request, int $tokenId): JsonResponse {
    $token = $request->user()->tokens()->findOrFail($tokenId);
    $token->delete(); // Cascade supprime aussi token_metadata
    AuditLog::record($request->user(), 'auth.session_revoked', ['token_id' => $tokenId]);
    return response()->json(['ok' => true]);
}

// DELETE /api/v1/auth/sessions – Révoquer TOUTES les sessions (sauf la courante)
public function revokeAllSessions(Request $request): JsonResponse {
    $currentTokenId = $request->user()->currentAccessToken()->id;
    $request->user()->tokens()->where('id', '!=', $currentTokenId)->delete();
    return response()->json(['ok' => true]);
}
```

## 2FA — TOTP (compatible Google Authenticator)

2FA TOTP — activation + confirmation

```
// Librairie : composer require pragmarx/google2fa-laravel

// Activer le 2FA — POST /api/v1/auth/2fa/enable
public function enable2FA(Request $request): JsonResponse {
    $google2fa = app(Google2FA::class);
    $secret = $google2fa->generateSecretKey();
    $qrCodeUrl = $google2fa->getQRCodeUrl(
        'Leopardo RH', $request->user()->email, $secret
    );
    // Stocker temporairement (validé après confirmation)
    Cache::put('2fa_setup:' . $request->user()->id, $secret, 300);
    return response()->json(['qr_code_url' => $qrCodeUrl, 'secret' => $secret]);
}

// Confirmer le 2FA — POST /api/v1/auth/2fa/confirm
public function confirm2FA(Request $request): JsonResponse {
    $secret = Cache::get('2fa_setup:' . $request->user()->id);
    $valid = app(Google2FA::class)->verifyKey($secret, $request->code);
    if (!$valid) return response()->json(['error_code' => 'INVALID_2FA_CODE'], 422);
    // Générer les codes de secours (one-time backup codes)
    $backupCodes = collect(range(1,8))->map(fn() => Str::upper(Str::random(10)))->toArray();
    $request->user()->update([
        'two_fa_enabled' => true,
        'two_fa_secret' => Crypt::encryptString($secret),
        'two_fa_backup_codes' => Crypt::encryptString(json_encode($backupCodes)),
    ]);
    AuditLog::record($request->user(), 'auth.2fa_enabled');
    return response()->json(['backup_codes' => $backupCodes]);
}
```



## CH.06

## SYSTÈME D'INVITATIONS

Inviter par email · Flow complet · Sécurité · Limite plan

06

## Email d'invitation — template Laravel

```
InvitationMail.php — email d'invitation
// resources/views/emails/invitation/fr.blade.php
// Même structure pour ar/tr/en (voir doc Multilinguisme)

// Sujet : '{{ $inviterName }} vous invite à rejoindre {{ $companyName }} sur Leopardo RH'

// Corps (Markdown email) :
// Bonjour,
// '{{ $inviterName }} vous a invité(e) à rejoindre l'équipe de {{ $companyName }}
// en tant que {{ __('roles.'.$role) }}'.
//
// Cliquez sur le bouton ci-dessous pour créer votre compte.
// Cette invitation expire le {{ $expiresAt->format('d/m/Y') }}.
//
// [BOUTON] Créer mon compte → {{ config('app.frontend_url') }}/invitation?token={{ $token }}
//
// Si vous ne connaissez pas {{ $companyName }}, ignorez cet email.

// app/Mail/InvitationMail.php
class InvitationMail extends Mailable {
    public function envelope(): Envelope {
        // Langue = langue de la company, pas forcément du destinataire
        App::setLocale($this->invitation->company->language);
        return new Envelope(subject: __('emails.invitation_subject', [
            'name' => $this->invitation->invited_by_name,
            'company' => $this->invitation->company->name,
        ]));
    }
}
```

## Limites du plan sur les invitations

| Plan       | Employés max | Rôles qu'on peut inviter                | Délai expiration invitation |
|------------|--------------|-----------------------------------------|-----------------------------|
| Trial      | 5 employés   | Employé uniquement                      | 3 jours                     |
| Starter    | 20 employés  | Employé + Superviseur                   | 7 jours                     |
| Business   | 100 employés | Tous les rôles                          | 14 jours                    |
| Enterprise | Illimité     | Tous les rôles + rôles custom (Phase 2) | 30 jours                    |

*Le check de limite plan s'effectue dans le middleware CheckPlanLimit avant la création de l'invitation — pas après. Si la company a atteint sa limite d'employés, la requête est rejetée avec PLAN\_EMPLOYEE\_LIMIT\_REACHED (402) avant même de créer le token d'invitation.*

**CH.07 PROGRESSIVE DISCLOSURE & STADES**

LearningStageService · Leo intégré · Adaptation interface

07

*Décision de conception clé de l'APV v2 : le stade d'apprentissage est calculé par LearningStageService depuis employees.metadata.app\_actions\_count. Il est retourné dans /api/v1/auth/me. Flutter et Blade l'utilisent pour décider ce qui est visible. Un seul service, une seule source de vérité.*

**LearningStageService — implémentation**

```
LearningStageService.php + réponse auth/me
// app/Services/LearningStageService.php
class LearningStageService {
    const STAGES = [
        'discovery' => ['min' => 0, 'max' => 9], // J0-J2, < 10 actions
        'routine' => ['min' => 10, 'max' => 49], // J3-J14
        'mastery' => ['min' => 50, 'max' => 199], // J15-J30
        'expert' => ['min' => 200, 'max' => 499], // J31+
        'ambassador' => ['min' => 500, 'max' => PHP_INT_MAX], // J60+
    ];

    public function forEmployee(Employee $emp): string {
        $count = (int)($emp->metadata['app_actions_count'] ?? 0);
        foreach (self::STAGES as $stage => $range) {
            if ($count >= $range['min'] && $count <= $range['max']) return $stage;
        }
        return 'ambassador';
    }

    // Incrémentation après chaque action réussie (appelé par middleware)
    public function increment(Employee $emp): void {
        $current = (int)($emp->metadata['app_actions_count'] ?? 0);
        $emp->update(['metadata->app_actions_count' => $current + 1]);
        // Mise à jour en batch toutes les 5 actions pour réduire les écritures
    }
}

// Retourné dans GET /api/v1/auth/me
'user' => [
    'id', 'name', 'email', 'role', 'manager_role',
    'learning_stage' => $stageService->forEmployee($employee),
    'language' => $employee->preferred_language ?? $company->language,
    'two_fa_enabled' => $employee->two_fa_enabled,
    'plan' => $company->plan->name,
    'features' => $company->plan->features, // JSONB features du plan
]
```

**Ce que chaque stade révèle dans l'interface**

| Stade         | Condition       | Flutter révèle...                                                                     | Blade révèle...                                         | Leo se comporte...                                                              |
|---------------|-----------------|---------------------------------------------------------------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------|
| ■ Découverte  | < 10 actions    | 3 chips max · Formulaires simplifiés · 0 filtre avancé · Checklist onboarding visible | Menu simplifié · Widgets essentiels · Onboarding banner | Guide actif : explique chaque écran au premier accès. Morning brief si alertes. |
| ■ Routine     | 10-49 actions   | + Filtres basiques · + Historique · + Badge progression profil                        | + Filtres dropdown · + Export simple                    | Réponses directes. Morning brief si alertes actives.                            |
| ■ Maîtrise    | 50-199 actions  | + Exports CSV · + Raccourcis (long press) · + Config avancée modules                  | + Export CSV · + ■K/■N raccourcis · + Config avancée    | Réponses courtes et précises. Propose des optimisations.                        |
| ■ Expert      | 200-499 actions | Tout ce que le plan autorise · Personnalisation chips (Phase 2)                       | Tout · Dashboard analytique si Enterprise               | Mode ressource. Répond aux questions pointues.                                  |
| ■ Ambassadeur | 500+ actions    | + Badge discret · + Accès bêta features en avant-première                             | + Programme parrainage actif · + NPS badge              | Mode pair. Confirme les best practices.                                         |

CH.08

PLANS & FEATURES — ARCHITECTURE EXTENSIBLE

JSONB standardisé · Feature flags · Upgrade/downgrade

08

Structure standardisée du JSONB features dans plans

La colonne plans.features est un JSONB libre. Le problème actuel : les clés ne sont pas documentées et chaque module en ajoute de nouvelles sans coordination. La v2 standardise toutes les clés en une seule source de vérité.

```
plans.features JSONB v2 — structure officielle
-- Structure officielle plans.features JSONB v2
-- (Mise à jour via migration + seeder)

UPDATE plans SET features = '{
  "max_employees": 20,           -- NULL = illimité
  "max_cameras": 0,             -- 0 = module désactivé
  "max_finance_docs_monthly": 0, -- 0 = module désactivé, NULL = illimité
  "modules": ["attendance", "employees"], -- Modules actifs
  "attendance": true,           -- Module Pointage
  "finance": false,             -- Module Finance
  "cameras": false,             -- Module Caméras
  "leave_management": true,     -- Gestion congés
  "tasks": false,              -- Module Tâches
  "salary_advances": false,    -- Avances sur salaire
  "payroll": false,            -- Module Paie (Phase 4)
  "api_access": false,         -- API publique (Enterprise Phase 2)
  "custom_roles": false,      -- Rôles custom (Enterprise Phase 2)
  "multi_company": false,     -- Multi-sociétés (Enterprise Phase 5)
  "leo_messages_daily": 5,    -- Limite messages Leo/jour (NULL = illimité)
  "leo_morning_brief": false, -- Brief matinal Leo
  "advanced_reports": false,   -- Rapports analytiques avancés
  "biometric": false,         -- ZKTeco biométrique
  "gps_tracking": false,      -- GPS obligatoire au pointage
  "whitelabel": false,       -- Logo personnalisé (Enterprise)
  "sla_support": false,      -- Support SLA garanti (Enterprise)
}' WHERE name = 'starter';
```

Les 4 plans — tableau comparatif complet v2

| Feature            | Trial | Starter | Business | Enterprise |
|--------------------|-------|---------|----------|------------|
| Employés max       | 5     | 20      | 100      | Illimité   |
| Module Pointage    | ✓     | ✓       | ✓        | ✓          |
| Module RH & Congés | ✓     | ✓       | ✓        | ✓          |
| Module Finance     | —     | —       | ✓        | ✓          |

| Feature                | Trial       | Starter  | Business          | Enterprise   |
|------------------------|-------------|----------|-------------------|--------------|
| Module Caméras         | —           | —        | ✓                 | ✓            |
| Module Tâches          | —           | —        | ✓                 | ✓            |
| Module Paie            | —           | —        | —                 | ✓            |
| Invitations            | 3 max       | ✓        | ✓                 | ✓            |
| Leo IA — messages/jour | 5           | 15       | 50                | Illimité     |
| 2FA                    | Oui         | Oui      | Oui               | Oui          |
| Délégation temporaire  | —           | —        | ✓                 | ✓            |
| Rapports avancés       | —           | —        | ✓                 | ✓            |
| Rôles custom (Phase 2) | —           | —        | —                 | ✓            |
| API publique (Phase 2) | —           | —        | —                 | ✓            |
| Multitenancy           | Shared      | Shared   | Shared            | Schema isolé |
| Support                | Email       | Email    | Email prioritaire | SLA garanti  |
| Prix mensuel           | Gratuit 14j | 29€/mois | 79€/mois          | 199€/mois    |

**CH.09 AUDIT TRAIL & SÉCURITÉ**

Qui a fait quoi · Quand · Depuis où · Alertes automatiques

09

**Évolution de audit\_logs v2**

```
audit_logs v2 – nouvelles colonnes + actions obligatoires

-- Migration : enrichir audit_logs avec les données de sécurité
ALTER TABLE audit_logs ADD COLUMN IF NOT EXISTS
  ip_address      INET          NULL, -- IP de l'acteur au moment de l'action
  user_agent      TEXT          NULL, -- Browser/app de l'acteur
  country         CHAR(2)       NULL, -- GeoIP2 – pays de l'action
  city            VARCHAR(100)  NULL, -- Ville de l'action
  session_token_id BIGINT       NULL; -- Quel token Sanctum a effectué l'action

-- Actions qui doivent TOUJOURS être auditées :
-- auth.login           – chaque connexion réussie
-- auth.login_failed    – chaque échec de connexion
-- auth.logout          – déconnexion
-- auth.session_revoked – révocation d'une session
-- auth.2fa_enabled     – activation du 2FA
-- auth.2fa_disabled    – désactivation du 2FA
-- auth.password_changed – changement de mot de passe
-- employee.created      – création d'un employé
-- employee.updated      – modification d'un employé
-- employee.archived     – archivage d'un employé
-- employee.suspended    – suspension d'un compte
-- invitation.created     – invitation envoyée
-- invitation.accepted    – invitation acceptée
-- attendance.manual_edit – correction de pointage
-- finance.invoice_sent  – envoi d'une facture
-- delegation.created    – création d'une délégation
-- plan.upgraded         – changement de plan
```

**AuditLogger — helper centralisé**

AuditLogger.php — helper centralisé

```
// app/Services/AuditLogger.php
```

```
class AuditLogger {
    public static function record(
        Employee|SuperAdmin|null $actor,
        string $action,
        array $context = [],
        ?Request $request = null
    ): void {
        $req = $request ?? request();
        AuditLog::create([
            'actor_type'      => $actor instanceof SuperAdmin ? 'super_admin'
                           : ($actor ? 'employee' : 'system'),
            'actor_id'       => $actor?->id,
            'actor_name'     => $actor?->name ?? 'System',
            'action'         => $action,
            'table_name'     => explode('.', $action)[0],
            'context'        => $context,
            'ip_address'     => $req?->ip(),
            'user_agent'     => $req?->userAgent(),
            'country'        => app(GeoLocationService::class)->locate($req?->ip())['country_code'] ?? null,
            'session_token_id'=> $req?->user()?->currentAccessToken()?->id,
        ]);
    }
}
```

```
// Usage partout dans le code :
```

```
AuditLogger::record($request->user(), 'employee.suspended',
    ['employee_id' => $employee->id, 'reason' => $request->reason]);
```

## Alertes de sécurité automatiques

| Événement                         | Seuil                                   | Action automatique                                            | Canal                                     |
|-----------------------------------|-----------------------------------------|---------------------------------------------------------------|-------------------------------------------|
| Connexions échouées               | 5 en 30 min sur le même email           | Verrouiller le compte 2h + notifier le manager                | Email manager + Slack admin               |
| Connexion depuis un nouveau pays  | 1 fois                                  | Notifier l'utilisateur + créer une tâche de vérification      | Email utilisateur                         |
| Révocation de toutes les sessions | Quand un admin révoque tout             | Notifier l'utilisateur par email                              | Email utilisateur                         |
| Changement de mot de passe        | Chaque changement                       | Notifier l'utilisateur + invalider toutes les autres sessions | Email + déconnexion forcée autres devices |
| Activation 2FA                    | Chaque activation                       | Sauvegarder les backup codes + notifier                       | Email avec backup codes                   |
| Invitation non acceptée           | Après 6 jours (1 jour avant expiration) | Relance email automatique                                     | Email invité                              |



| Événement      | Seuil               | Action automatique  | Canal         |
|----------------|---------------------|---------------------|---------------|
| Compte inactif | 30 jours sans login | Notifier le manager | Email manager |

CH.10

## TOUTES LES TÂCHES — USR-01 À USR-48

4 sprints · DB · Laravel · Flutter · Priorités

10

Ces 48 tâches modernisent la gestion des utilisateurs sans casser l'existant. Toutes les migrations sont additive (ajout de colonnes, nouvelles tables) — jamais de DROP sur les tables existantes.

## SPRINT 1 — Migrations &amp; Fondations DB

Nouvelles tables · Colonnes · Index · Seeders · ~5 j-dev

● USR-01

**Migration : columns v2 employees**  
Ajouter preferred\_language, metadata\_JSONB, two\_fa\_enabled, two\_fa\_secret, two\_fa\_backup\_codes, password\_changed\_at, failed\_login\_count, locked\_until. Toutes backward-compatible (NOT NULL avec DEFAULT ou NULL). Critique

● USR-02

**Migration : table invitations (public)**  
Créer la table invitations dans le schéma public (voir Ch.02). Index unique sur (company\_id, email) avec WHERE sur invitations actives. Test rollback. Critique

● USR-03

**Migration : table token\_metadata**  
Créer token\_metadata liée à personal\_access\_tokens. Vue active\_sessions. Index sur last\_used\_at DESC. Critique

● USR-04

**Migration : table delegations (tenant)**  
Créer delegations dans le schéma tenant (shared + schema). CHECK constraints dates et not\_self. Index sur delegate\_id + is\_active. Haute

● USR-05

**Migration : table password\_history (tenant)**  
Créer password\_history dans le schéma tenant. Index sur (employee\_id, created\_at DESC). 0,25 j · Haute

● USR-06

**Migration : audit\_logs v2 colonnes**  
Ajouter ip\_address, user\_agent, country, city, session\_token\_id aux audit\_logs existants. Index sur (ip\_address) pour les recherches de sécurité. Critique

● USR-07

**Seeder plans features v2 standardisé**  
Mettre à jour tous les plans existants avec le JSONB features standardisé (voir Ch.08). Idempotent. Vérifier que les features existantes ne sont pas perdues. Critique

● USR-08

**Modèle Eloquent Invitation**  
app/Models/Invitation.php : SoftDeletes, scopes (active, expired, revoked), méthodes isExpired(), isRevoked(), isAccepted(). Relation belongsTo Company. Critique

● USR-09

**Modèle Eloquent TokenMetadata**  
app/Models/TokenMetadata.php : Relation belongsTo PersonalAccessToken. Scope active. Observer pour mettre à jour last\_used\_at à chaque requête. Critique

● USR-10

**Modèle Eloquent Delegation**

app\Models\Delegation.php : Scope active (is\_active=true AND starts\_at<=today AND ends\_at>=today). Haute  
 Laravel  
 Méthodes isActive(), covers(permission).

## SPRINT 2 — Backend

### Laravel — Auth & Invitations

Login v2 · 2FA · Sessions · Invitations · Délégation · ~10 j-dev

● USR-11

**AuthController@login.v2**

Rate limiting par email (Str::lower). Vérification locked\_until. Vérification status (suspended/archived). 1 j · Critique  
 Laravel  
 POST /api/v1/auth/login. Incrémentation failed\_login\_count. Verrouillage après 10 échecs. Retour pre\_token si 2FA activé.

● USR-12

**AuthController@verify2FA**

POST /api/v1/auth/2fa/verify. Vérifier le code TOTP depuis le pre\_token en cache. Accepter aussi les 0,5 j · Critique  
 Laravel  
 backup codes (one-time use). Émettre le vrai token Sanctum si validé.

● USR-13

**AuthController@enable2FA + confirm2FA**

POST /api/v1/auth/2fa/enable. Générer le QR code URL. Vérifier le premier code. Stocker 0,75 j · Haute  
 Laravel  
 secret chiffré. Générer 8 backup codes. Audit log.

● USR-14

**SessionController@CRUD**

DELETE /auth/sessions/{id} (révocation ciblée). DELETE /auth/sessions (révocation tout sauf current). Audit log sur chaque action. 0,75 j · Haute  
 Laravel

● USR-15

**TokenMetadata enrichment middleware**

Middleware UpdateTokenMetadata : après chaque requête authentifiée réussie, mettre à jour 0,5 j · Critique  
 Laravel  
 last\_used\_at, last\_ip, last\_country dans token\_metadata. Asynchrone via job si performance concern.

● USR-16

**InvitationController — créer et envoyer**

POST /api/v1/employees/invite. Vérifier limite plan (CheckPlanLimit middleware). Vérifier email pas déjà 1 j · Critique  
 Laravel  
 actif. Créer invitation. Envoyer InvitationMail dans la langue de la company. Audit log.

● USR-17

**InvitationController — accepter l'invitation**

POST /api/v1/invitations/{token}/accept. Vérifier token valide/non-expiré. DB::transaction : créer employé 1 j · Critique  
 Laravel  
 + user\_lookup + token Sanctum + token\_metadata. Retourner auth response complète.

● USR-18

**InvitationController — révoquer / lister**

DELETE /api/v1/invitations (liste des invitations actives/expirées). DELETE /api/v1/invitations/{id} (révoquer 0,5 j · Haute  
 Laravel  
 une invitation non encore acceptée). Audit log.

● USR-19

**LearningStageService**

Service singleton (voir UR-07). Calcul du stade depuis metadata.app\_actions\_count. Incrément via 0,5 j · Critique  
 Laravel  
 ActionTrackerMiddleware (batch toutes les 5 actions). Méthode shouldReveal(feature, stage).

● USR-20

**GET /api/v1/auth/me enrichi**

Retourner user, me, learning\_stage + language + two\_fa\_enabled + plan + features + delegations 0,5 j · Critique  
 Laravel  
 actives. Cette réponse est le point de départ de toute la session Flutter.

● USR-21

**DelegationController@CRUD**

GET /api/v1/delegations. Vérifier que le delegator a le droit de déléguer. Vérifier que le 0,75 j · Haute  
 Laravel  
 delegate existe dans la même company. Dates valides. Audit log.

● USR-22

**DelegationService — résolution permissions**

Service canApprove(employee, permission) qui vérifie le rôle direct ET les délégations actives. Intégrer dans toutes les Gates/Politiques existantes (leave, expense, attendance). Tester avec les 7 rôles.

0,5 j · Haute

● USR-23

**PasswordController — changement sécurisé**

POST /api/v1/auth/password/change. Vérifier ancien MDP. Vérifier complexité nouveau MDP (min 8 chars, 1 majuscule, 1 chiffre). Vérifier pas dans les 5 derniers (password\_history). Révoquer toutes les autres sessions. Notifier par email.

0,5 j · Haute

● USR-24

**Audit logger helper + enrichissement**

Créer AuditLogger.record() (voir Ch.09). Ajouter ip\_address + user\_agent + country sur toutes les actions existantes. Vérifier que les actions listées en Ch.09 sont bien auditées.

0,75 j · Critique

## SPRINT 3 — Flutter — Interface utilisateur

Sessions · 2FA · Invitations · Stades · Settings · ~10 j-dev

- **USR-25**

**AuthBloc v2 — stade + features**

Flutter

Étendre AuthBloc pour stocker learning\_stage, features (plan), two\_fa\_enabled, preferred\_language. Recharger depuis /auth/me au démarrage. Exposer via AuthState à tous les modules.

1 j · Critique
- **USR-26**

**Login screen — 2FA step**

Flutter

Si !two\_fa\_enabled || requires\_2fa=true → afficher l'étape 2FA. TextField 6 chiffres (OTP style). Bouton 'Utiliser un code de secours'. Timer de 30s pour le code TOTP affiché. Submit → POST /auth/2fa/verify.

0,75 j · Critique
- **USR-27**

**Sessions screen — liste et révocation**

Flutter

Écran 'Mes appareils connectés' dans ProfileScreen. Liste des sessions avec device\_name, platform, (icône), last\_used\_at (heure relative), pays. Bouton révoquer (avec confirmation). Bouton 'Révoquer tous les autres'.

0,75 j · Haute
- **USR-28**

**Settings — activer/désactiver 2FA**

Flutter

Dans SettingsScreen : toggle 2FA. Activer → afficher QR code (qr\_flutter package) + champ code de confirmation. Désactiver → demander le MDP actuel. Afficher les backup codes après activation.

0,75 j · Haute
- **USR-29**

**Settings — changement de langue**

Flutter

Dans SettingsScreen : selecteur de langue (4 options avec noms natifs). Tap → PATCH /profile/language → mise à jour locale immédiate (MaterialApp.router reloads avec nouvelle locale).

0,25 j · Haute
- **USR-30**

**Progressive disclosure — LearningStageProvider**

Flutter

LearningStageProvider (Riverpod). shouldReveal(feature, stage) → bool. Wraps les features avancées dans les modules. Lu depuis AuthState.learningStage. Pas de restart requis au changement de stade.

0,75 j · Critique
- **USR-31**

**Onboarding checklist Flutter**

Flutter

Onboarding checklist widget visible si stage='discovery'. 5 étapes. Progress bar. Dismiss une fois toutes les étapes complétées. Persisté dans metadata.onboarding\_completed via PATCH /profile/metadata.

0,75 j · Haute
- **USR-32**

**ProfileScreen — informations utilisateur**

Flutter

Afficher : nom, email, role (badge couleur), stade apprentissage (badge), plan de la company, langue. Bouton modifier photo/téléphone. Bouton changer MDP.

0,5 j · Haute
- **USR-33**

**Invitation accept screen — web + Flutter**

Flutter

Page web (landing page publique) : afficher les infos de l'invitation, formulaire nom + MDP. Validation côté client avant soumission. Sur Flutter : deep link depuis l'email → InvitationAcceptScreen si app installée.

1 j · Critique
- **USR-34**

**Delegations screen (manager uniquement)**

Flutter

Visible uniquement si role=manager ET plan >= Business. Lister les délégations actives (donné et reçues). Créer une délégation : sélectionner employé, permission\_scope, dates. Confirmer avant soumission.

0,75 j · Haute
- **USR-35**

**Erreurs auth — messages humains**

Flutter

ErrorMessages.dart v2 : ACCOUNT\_LOCKED (avec durée), PLAN\_EMPLOYEE\_LIMIT\_REACHED, INVITATION\_EXPIRED, INVALID\_2FA\_CODE, INVITATION\_ALREADY\_ACCEPTED. Tous en FR/AR/TR/EN via ARB.

0,3 j · Critique

● USR-36

**Tests widget — écrans auth**

Tests Widget Flutter : LoginScreen (succès, 2FA step, compte suspendu), SessionsScreen (liste + 1 j · Haute révocation), OnboardingChecklist (progression), LearningStageProvider (transitions). Coverage > 70%.

**SPRINT 4 — Polish, sécurité & tests**

Alertes sécurité · Rate limiting · Tests · Audit · ~6 j-dev

● USR-37

**Rate limiting par email (brute force)**

Tests : Web Flutter : LoginScreen (5 tentatives / 30min. Après 10 tentatives cumulatives : locked\_until +2h. Test : 6 tentatives sur le même email → 429. 11 tentatives → 403 locked. 0,5 j · Critique

● USR-38

**Alertes sécurité automatiques — n8n**

Configurer les 3 alertes de sécurité (voir Ch.09) via events Laravel → webhook n8n → emails/Slack. 0,75 j · Haute

● USR-39

**Politique de mot de passe — validation**

Laravel : password rules : min 8 chars, 1 majuscule, 1 chiffre, pas dans les 5 derniers. Appliquer sur 0,5 j · Haute registration, invitation accept, password change. Messages d'erreur traduits.

● USR-40

**Tests Pest — Auth v2 complets**

Tests : login normal (201), login 2FA (200 requires\_2fa), login compte suspendu (403), brute force 1,5 j · Critique protection (429 après 5), invitation flow complet, délégation résolution permissions, stade calcul.

● USR-41

**Tests Pest — Invitations**

Tests : create invitation (201), invitation déjà active (422), accepter invitation valide (201 + employee crée + 0,75 j · Haute user\_lookup), accepter invitation expirée (422), accepter invitation déjà acceptée (422).

● USR-42

**Tests Pest — Isolation tenant**

Tests : employee company A ne peut pas voir les sessions de company B. Super admin ne peut pas 0,5 j · Critique accepter une invitation tenant. Délégation cross-company impossible (404).

● USR-43

**Purge automatique données expirées**

Configure artisan users:cleanup : purger invitations expirées > 30j, tokens Sanctum expirés, sessions 0,25 j · Haute non utilisées > 90j, password\_history > 5 entrées par employé. Planifier en cron.

● USR-44

**RGPD — Export données utilisateur**

GET /api/v1/profile/export-data : retourner en JSON toutes les données personnelles d'un employé (profil, 0,5 j · Haute pointages, congés, audit logs le concernant). Droit à la portabilité RGPD.

● USR-45

**RGPD — Anonymisation à l'archivage**

À l'archivage d'un employé : conserver les données opérationnelles (pointages, congés) mais anonymiser 0,5 j · Haute : name → 'Employé Archivé', email → hash, phone → null, national\_id → null. user\_lookups : conserver l'email réservé.

● USR-46

**Documentation API — user management**

Mettre à jour /2\_API\_CONTRACTS\_COMPLETE.md avec tous les nouveaux endpoints (invitations, 0,5 j · Haute sessions, 2FA, délégations, export RGPD). Schema OpenAPI pour chaque endpoint.

● USR-47

**ERD mis à jour — v2**

Mettre à jour /3\_ERD\_COMPLETE.md avec les nouvelles tables et colonnes v2. Diagramme ERD 0,25 j · Critique regeneré. Vérifier cohérence avec les migrations réellement appliquées.

● USR-48

PILOTAGE.md — mise à jour

Transversal

Marquer les features user management v2 comme livrées. Mettre à jour PROGRAM\_VERSION. Ajouter la gestion des invitations et des sessions dans la liste des features production-ready.

0.25 j Haute

| Sprint   | Thème                                | Tâches    | Effort    | Résultat                                         |
|----------|--------------------------------------|-----------|-----------|--------------------------------------------------|
| Sprint 1 | Migrations & Fondations DB           | 10        | ~5 j      | Nouvelles tables, colonnes enrichies, index      |
| Sprint 2 | Backend Laravel — Auth & Invitations | 14        | ~10 j     | Login v2, 2FA, sessions, invitations, délégation |
| Sprint 3 | Flutter — Interface utilisateur      | 12        | ~10 j     | 2FA UI, sessions screen, progressive disclosure  |
| Sprint 4 | Polish, sécurité & tests             | 12        | ~6 j      | Alertes, RGPD, tests complets, documentation     |
| TOTAL    | Architecture Gestion Utilisateurs v2 | 48 tâches | ~31 j-dev | Le socle rock-solid de Leopardo RH               |

## CH.11

## HORIZON PHASE 2 — PERMISSIONS GRANULAIRES

Rôles custom · Délégation fine · API publique · Multi-sociétés

11

*Ces features ne sont pas dans le scope de la v2 actuelle. Elles sont documentées ici pour guider les décisions d'architecture d'aujourd'hui. Aucune décision de la v2 ne doit fermer la porte à ces évolutions.*

## Rôles custom — Enterprise Phase 2

Un client Enterprise doit pouvoir créer son propre rôle 'Superviseur Chantier' avec exactement les permissions qu'il choisit — pas les 7 rôles prédéfinis. Architecture :

- **Table custom\_roles** (schéma tenant) : id, name, permissions JSONB (liste des actions autorisées), created\_by.
- **employees.custom\_role\_id** NULL FK → custom\_roles.id. Si custom\_role\_id est défini, les permissions du custom role s'appliquent à la place des permissions du manager\_role.
- **PermissionResolver** : service qui résout les permissions dans l'ordre : custom\_role → manager\_role → role. Le premier niveau qui a une réponse gagne.
- **Interface admin** : un manager principal peut créer des rôles custom depuis le dashboard web avec une matrice de permissions visuelle.

## API publique — Enterprise Phase 2

Un client Enterprise doit pouvoir connecter Leopardo RH à ses propres systèmes (ERP, comptabilité, SIRH). L'API publique est une extension de l'API actuelle avec des clés API dédiées.

```
api_keys — table pour l'API publique Phase 2
-- Table api_keys (schéma public)
CREATE TABLE api_keys (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id        UUID NOT NULL REFERENCES companies(id) ON DELETE CASCADE,
  name              VARCHAR(100) NOT NULL, -- 'Intégration ERP SAP'
  key_hash          VARCHAR(255) NOT NULL, -- Hash SHA-256 de la clé réelle
  scopes            JSONB NOT NULL,        -- ['attendance:read','employees:read']
  last_used_at      TIMESTAMPTZ NULL,
  expires_at        TIMESTAMPTZ NULL,      -- NULL = pas d'expiration
  created_by        INT NOT NULL,          -- employee_id du créateur
  created_at        TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
-- La clé réelle (Bearer api_LeopardoRH_XXXXXX) n'est jamais stockée
-- Elle est affichée une seule fois à la création → l'utilisateur doit la copier
```

## Multi-sociétés — Phase 5



Un holding qui gère 5 filiales doit voir les données consolidées de toutes ses entités depuis une seule interface. Architecture :

- **Table company\_groups** (schéma public) : id, name, parent\_company\_id. Une company peut appartenir à un groupe.
- **Super Manager de groupe** : un rôle spécial qui peut voir les KPIs consolidés de toutes les companies du groupe, sans accéder aux données RH individuelles.
- **Rapport consolidé** : endpoint GET /api/v1/group/dashboard qui agrège les métriques de toutes les companies du groupe (présences, absences, dépenses) avec drill-down possible.

*Ce document est la référence v2 de l'architecture gestion des utilisateurs de Leopardo RH. Il remplace et complète le document RBAC\_COMPLET.md existant sans le contredire. Il doit être lu en parallèle du document APV v2 (Architecture Produit Vivante) qui définit le paradigme global. Version 2.0 — Avril 2026 — Équipe Technique Leopardo RH.*